Development Roadmap

VIRA — Bike Spare Parts Billing & Business Management Software

This roadmap follows the Software Requirements Specification section by section, so every task below traces back to an exact requirement. Fifteen phases, each with a concrete “Done when” exit test — nothing moves to the next phase on the strength of code alone.

How the pieces fit together

Layer	Role
Electron window	Hosts the React screens (Home, New Bill, Daybook, ...) and the native print pipeline
React (HTTP, localhost:8080/api)	Shows and collects data only — it decides nothing
Spring Boot	Owns every rule and calculation: pricing, totals, Cash in Hand, Party Current Balance, numbering, reverse-then-apply updates
MySQL (local, via JDBC)	Stores bills, items, parties, suppliers, cash — including every deleted record, kept and flagged rather than removed

Cash in Hand and Party Current Balance are stored values, continuously maintained by a single reverse-then-apply update inside every create, edit, delete and restore transaction (SRS §15.3, §20.4) — never recomputed by scanning history on each read. A Recalculate Balances tool exists purely as an offline safety net, never as the normal calculation path.

Phase 0 — Project Setup

Repository, build pipeline, and a backend that boots against a fresh database

- Repository with a Spring Boot (Java) backend skeleton and a React/Vite frontend skeleton.
- MySQL running locally; Flyway (or equivalent) wired into the Spring Boot build for schema migrations.
- Basic CI (build + test) if desired.

Done when — ./mvnw spring-boot:run **(or equivalent) starts the backend against a fresh MySQL database created entirely from migrations, and the React dev server talks to it.**

Phase 1 — Database Schema

Every table in SRS §22, including the four new ones the stored-balance design needs

- item, part_category, bike, item_bike tables (SRS §22.1–22.2)
- party_item_rate, cash_item_rate tables (SRS §22.3)

- party table — including opening_balance, current_balance, is_deleted, deleted_at (SRS §22.4)
- bill, bill_item tables, including the snapshot columns, item_type_snapshot, line_no, is_edited / last_edited_at and is_deleted / deleted_at (SRS §22.5)
- payment_voucher table, including is_deleted / deleted_at (SRS §22.6)
- supplier, supplier_payment tables, both including is_deleted / deleted_at (SRS §22.7)
- shop_profile single-row table — all fields from SRS §22.8, including the new voucher_prefix and supplier_payment_prefix columns
- cash_in_hand single-row table: id, balance, last_updated_at (SRS §22.9)
- number_sequence table: sequence_name, next_value, seeded with one row each for CASH_BILL, PARTY_BILL, VOUCHER and SUPPLIER_PAYMENT (SRS §22.10)
- app_setting table: setting_key, setting_value, seeded with the five SRS §19.1 preference rows (SRS §22.11)
- Confirm foreign keys match the ON DELETE behaviour specified per table in SRS §22 (SET NULL for Item-linked snapshot references; CASCADE for rate overrides and bill lines; party_id / supplier_id kept as defensive SET NULL even though the application itself never hard-deletes those rows — SRS §22.12)
- Get the shop's actual starting values and store them: Opening Cash in Hand amount + effective date, seeded straight into cash_in_hand; Cash/Party/Voucher/Supplier-Payment bill prefixes and starting numbers, seeded into number_sequence (SRS §18.3)

NEW IN THIS PLAN — cash_in_hand, number_sequence and app_setting are new tables with no equivalent in an earlier live-calculation design. party, bill, payment_voucher, supplier and supplier_payment each gain is_deleted / deleted_at; party additionally gains opening_balance and current_balance.

Done when — A fresh MySQL database can be built from the migration history with one command, matches SRS §22 exactly, and the four seed rows (cash_in_hand, four number_sequence rows, five app_setting rows) exist immediately after migration.

Phase 2 — Backend Core: Items & Pricing

The one area of the system that keeps its original, permanent-delete design

- Entities/repositories: Item, PartCategory, Bike, ItemBike, PartyItemRate, CashItemRate
- Items API: list/search/filter, create, edit, permanent delete (cascades rate overrides, never touches historical BillItem snapshots — SRS §6)
- Validation: Item Type is locked (cannot change) once any PartyItemRate or CashItemRate row exists for that item (SRS §4.4, §23)
- PricingService implementing the precedence rules (SRS §7.4) and Permanent/This Bill Only (SRS §8)
- Unit tests: MRP calculation, Net Rate fallback, Party override, Cash override, Permanent vs. This-Bill-Only, Item Type lock once an override exists, and that deleting an Item never breaks an existing BillItem's display

Done when — Every pricing example in SRS §7–8 has a passing test, Items CRUD (including permanent delete) works end-to-end, and the Item Type lock test passes.

Phase 3 — Backend Ledger Core: Cash in Hand & Party Balance

The reverse-then-apply engine every later phase builds on

This phase exists ahead of Bills and Parties in the original plan because every transaction phase after this one depends on it — build the ledger primitives once, correctly, before anything calls them.

- CashInHand entity/repository, with a row-locking read (SELECT … FOR UPDATE / pessimistic write) exposed as lockForUpdate()
- Party.currentBalance column wired into the Party entity, with the same row-locking read exposed per party
- LedgerService with two primitives: adjustCash(delta) and adjustPartyBalance(partyId, delta) — every later service calls only these two methods to move money, never raw SQL updates
- NumberSequenceService: nextValue(sequenceName), incrementing inside the caller's own transaction so a number is never derived by scanning existing rows and is never issued twice (SRS §18.3)
- Unit tests: concurrent adjustCash calls never lose an update (simulate two near-simultaneous writes); NumberSequenceService never returns the same value twice under concurrent calls

NEW IN THIS PLAN — LedgerService and NumberSequenceService are the two foundational services this entire plan is built around. Every Bill, Payment Voucher and Supplier Payment service in Phases 4–5 calls these instead of touching cash_in_hand or party.current_balance directly.

Done when — A small test harness can fire fifty concurrent adjustCash(+1) calls and read back exactly +50 on cash_in_hand.balance, with no lost updates.

Phase 4 — Backend Transactions: Parties, Bills & Suppliers (Create)

Create-only for now — edit, delete and restore are Phase 5

- Party entity + API: create (including optional Opening Balance, seeding current_balance), list/search, Selected Party page data
- Supplier entity + API: create, list/search
- Bill + BillItem entities, with snapshot fields (including item_type_snapshot) populated at save time
- Bill-create endpoint: validation, line calculation, automatic nearest-Rupee round-off with manual override, grand total (SRS §10.8), bill number from NumberSequenceService, LedgerService.adjustCash or adjustPartyBalance called once, all inside a single DB transaction (SRS §24)
- Previous Balance / Total Due calculation for Party Bills, read directly from party.current_balance (SRS §10.9)

Done when — A Cash Bill and a Party Bill can each be created through the API with correct totals, the Party's current_balance is correct on the next read with no SUM query involved, and a new Party's current_balance exactly equals its Opening Balance before any bill is raised against it.

Phase 5 — Backend Universal Edit, Delete & Restore

The reverse-then-apply pattern, applied uniformly across every transaction type

Edit (PUT)

- PUT /api/bills/{id}, PUT /api/payment-vouchers/{id}, PUT /api/supplier-payments/{id} — each: lock the row, reverse its currently-stored effect via LedgerService using the values presently on the record, apply the submitted edits, recompute totals, apply the new effect, set is_edited / last_edited_at (SRS §20.3–20.4)
- Reject edits where the record itself, or its linked Party/Supplier, is currently deleted

Delete (soft)

- DELETE /api/bills/{id}, DELETE /api/payment-vouchers/{id}, DELETE /api/supplier-payments/{id} — reverse the stored effect via LedgerService, set is_deleted = true and deleted_at = now; the row itself is never removed (SRS §20.6)
- DELETE /api/parties/{id} — blocked unless current_balance = 0; on success, set is_deleted / deleted_at only. No LedgerService call: a Party's own delete never touches a balance, because it never held one directly
- DELETE /api/suppliers/{id} — no balance guard; same is_deleted / deleted_at pattern
- DELETE /api/items/{id} — unchanged from Phase 2: the one real, permanent DELETE in the system

Restore

- POST /api/bills/{id}/restore, /api/payment-vouchers/{id}/restore, /api/supplier-payments/{id}/restore — re-apply the record's stored effect via LedgerService exactly as a fresh create would, then clear is_deleted / deleted_at
- POST /api/parties/{id}/restore, /api/suppliers/{id}/restore — clear is_deleted / deleted_at only; no LedgerService call
- Availability rule shared by every Edit/Delete endpoint and by the frontend: a Bill or Payment Voucher is editable/deletable only when it is itself active AND its linked Party (if any) is active; a Supplier Payment likewise requires its Supplier to be active

Recalculate Balances

- POST /api/maintenance/recalculate-balances — recomputes cash_in_hand and every party.current_balance from scratch, summing only active records per the SRS §15.1 formula, compares to the stored value, reports any mismatch, and — only on explicit confirmation — overwrites the stored value (SRS §20.9)

Test scenarios to cover:

- Edit a Cash Bill's quantity → Cash in Hand updates correctly on next read
- Edit a Bill from Cash to Party (or vice versa) → the old side is reversed and the new side is applied correctly, with no special-case code needed
- Edit a Party Bill's Party → old Party's balance drops, new Party's balance rises
- Delete a Cash Bill → Cash in Hand drops by exactly that bill's amount, and the bill remains visible, tagged Deleted, with every original line item intact
- Delete a Party Bill → that Party's balance drops by exactly that bill's amount
- Restore a deleted Cash Bill → Cash in Hand rises back by exactly that bill's amount, and its Edit/Delete icons work again
- Edit a Payment Voucher's mode from Online to Cash → Cash in Hand unaffected (mode never splits the figure)
- Delete a Supplier Payment → Cash in Hand rises back by exactly that amount
- Attempt to delete a Party with a non-zero balance → rejected

- Delete a Party with a zero balance → its existing Bills remain visible, but their Edit and Delete endpoints now reject with a clear “party is deleted” reason
- Restore that Party → its Bills' Edit/Delete endpoints work again, and no balance changed as a result of either the delete or the restore
- Delete an Item that appears on old bills → old bills still display correctly from their BillItem snapshots (unchanged from Phase 2)
- Run Recalculate Balances against a small dataset with a deliberately-corrupted stored value → the mismatch is detected and correctly reported

Done when — Every scenario above passes, matching SRS §20 and §26's acceptance criteria, and the reverse-then-apply pattern in LedgerService is the only code path that ever changes cash_in_hand.balance or party.current_balance — confirmed by searching the codebase for any other write to those two columns.

Phase 6 — Backend Remaining Views

Read APIs for every other screen in the menu

- Daybook query API: filters including “Include deleted bills”, summary figures computed over active bills only (SRS §12)
- Dashboard KPIs + recent-activity queries, KPIs computed over active records only (SRS §16)
- Part Search query (SRS §17)
- Shop Profile API, including the four numbering prefixes (SRS §18)
- Settings API — reading and writing the five app_setting rows (SRS §19.1)

Done when — Every screen in the menu has a working read API behind it, and the Daybook/Dashboard summary figures are verified to exclude a deliberately-deleted test bill.

Phase 7 — Frontend Foundation

Shared components built once, including the three-state Edit / Delete / Restore control

- Routing for all menu items (SRS §3)
- API client (fetch wrapper, error handling)
- Shared table, form, modal, and money/date-formatting components
- One reusable Edit modal and one reusable Delete confirmation dialog per record type (Bill, Payment Voucher, Supplier Payment, Party, Supplier), built once here so every later screen just plugs them in (SRS §20.5, §25.2)
- One reusable row-action control that renders Edit + Delete icons for an active record, or a single Restore icon plus a muted “Deleted” badge for a deleted one — driven purely by the record's is_deleted flag, so no screen has to re-implement that branching logic
- A shared “Revised” / “Deleted” badge component that can render either, both, or neither next to a record

Done when — The shared Edit, Delete and Restore components exist and can open from a stub row in all three states (active / deleted / just-restored), even before every screen wires them in.

Phase 8 — Frontend Master Screens

Items, Parties, Suppliers and the screens that configure the whole system

- Items screen: list/search/filter/add/edit/permanent-delete (SRS §6)
- Parties screen: active-party list plus a Deleted Parties tab; Selected Party page with Bills / Payment History / Item Rates tabs, every row wired to the shared row-action control; Party delete blocked in the UI when balance ≠ ₹0, with the reason shown inline (SRS §9, §20.2)
- New Party form includes the optional Opening Balance field, clearly labelled as set-once (SRS §9.1)
- Suppliers screen: active-supplier list plus a Deleted Suppliers tab; Supplier page with Payment History wired to the shared row-action control (SRS §14, §20.2)
- Shop Profile screen, including all four bill/voucher/payment numbering prefixes and print settings (SRS §18)
- Settings screen, including the Recalculate Balances action with its mismatch report (SRS §19)

Done when — Every master screen matches its SRS fields, every Bill/Voucher/Payment row visible on these screens shows the correct Edit/Delete/Restore state, and a deleted Party's page correctly locks its own Bills tab while still displaying every row.

Phase 9 — Frontend New Bill

The primary operational screen, and its own Edit-mode

- Header, F2 item search, item entry, duplicate-line-adds-new-row behaviour (SRS §10.2–10.3)
- 15-row grid with internal scroll + auto-focus on new line (SRS §10.7)
- Permanent/This Bill Only popup (SRS §8)
- Totals, automatic + manual round-off, Previous Balance / Total Due for Party Bills (SRS §10.8–10.9), reading Previous Balance straight from the Party's current stored balance
- Confirm this same screen works as “Edit Bill” in edit mode (SRS §20.5) — should already follow from Phase 7's shared component, and is disabled entirely for a deleted bill

Done when — New Bill matches SRS §10 exactly, doubles as the Edit-Bill screen with no duplicated code, and attempting to open a deleted bill in edit mode is blocked with a clear message pointing at the Restore action instead.

Phase 10 — Frontend Daybook / Payment Vouchers / Supplier Payments

Every list screen, with deleted-record visibility built in

- Daybook: filters including “Include deleted bills”, summary, table with the Status column (Revised / Deleted badges), view/print, row-action control on every row (SRS §12, §20.2)
- Payment Vouchers screen: list + new + row-action control on every row (SRS §13, §20.2)
- Supplier Payment entry reused in edit mode, with the row-action control (SRS §14.3, §20.2)
- Delete confirmation dialog text matches SRS §20.6 exactly per record type — the Bill/Voucher/Payment wording differs from the Party/Supplier wording, and both are visually distinct from the Edit action (SRS §25.2)

Done when — Every location listed in SRS §20.2 has a working Edit action, a working Delete action, and — once deleted — a working Restore action, and the Daybook

summary figures are confirmed to stay unchanged when a bill in the visible list is toggled to show as deleted.

Phase 11 — Frontend Dashboard, Part Search, Printing

The remaining screens, plus the print pipeline's Revised/Deleted tags

- Dashboard KPIs, charts, recent-activity widgets — rows wired to the shared row-action control (SRS §16, §20.2)
- Part Search screen (SRS §17)
- Bill print layout via the Electron print pipeline, driven by Shop Profile settings, including Previous Balance/Total Due on Party Bills and the Revised and/or Deleted tags on the printed view (SRS §11)

Done when — Dashboard recent-activity rows are editable/deletable/restorable as appropriate, Part Search works, and a bill prints correctly both after being edited and after being deleted (showing both tags together where both apply).

Phase 12 — Testing Against the SRS

A full pass against the acceptance criteria, with particular weight on the ledger

- Walk every line of SRS §26 (Acceptance Criteria) and §27 (Final Business Rule Reference) against the running app
- Dedicated regression pass on Phase 5's scenario list, run again here end-to-end through the UI rather than only against the API
- Have a second, independent review of the app against the SRS
- Fix whatever gaps are found

Done when — Every acceptance-criteria line is checked off against the running application, not just the code, and the Phase 5 scenario list passes a second time end-to-end through the UI.

Phase 13 — Electron Packaging

Fully offline, on a real Windows machine

- Electron shell starts Spring Boot, checks for local MySQL, and loads the React build (SRS §2)
- Verify the whole app works with the PC's internet disabled

Done when — The app runs fully offline on a Windows machine from the Electron shell, including Recalculate Balances and every Restore action.

Phase 14 — Installer & Handover

Getting it onto the shop's PC without a developer in the room

- Windows build produces the installer (.exe)
- Backup/restore for the local MySQL database — confirm scope with the shop owner first
- Short user guide for day-to-day use, including one page specifically on what Delete now means (marked and recoverable, not erased) and how to use Restore

Done when — A shop owner can install and run VIRA on a clean Windows PC without a developer present, and can correctly explain — from the user guide alone — what happens when they delete a bill or a party.